

Bilinear#

[https://pytorch.org/docs/stable/generated/torch.nn.modules.linear.Bilinear.h](https://pytorch.org/docs/stable/generated/torch.nn.modules.linear.Bilinear.html)

Description:

Applies a bilinear transformation to the incoming data: $y = x_1^T A x_2 + b$. $y = x_1^T A x_2 + b$. in1_features (int) – size of each first input sample, must be > 0 in2_features (int) – size of each second input sample, must be > 0 out_features (int) – size of each output sample, must be > 0 bias (bool) – If set to False, the layer will not learn an additive bias.
Default: True

Function Signature

```
class torch.nn.modules.linear.Bilinear(in1_features,  
in2_features, out_features, bias=True, device=None,  
dtype=None)[source]#
```

Parameters

Shape:

Variables

```
extra_repr()[source]#
```

Returns:

Tensor

Code Examples

```
>>> m = nn.Bilinear(20, 30, 40)
>>> input1 = torch.randn(128, 20)
>>> input2 = torch.randn(128, 30)
>>> output = m(input1, input2)
>>> print(output.size())
torch.Size([128, 40])
```

Detailed Documentation

Documentation

Applies a bilinear transformation to the incoming data: $y = x_1^T A x_2 + b$.

in1_features (int) – size of each first input sample, must be > 0

in2_features (int) – size of each second input sample, must be > 0

out_features (int) – size of each output sample, must be > 0

bias (bool) – If set to False, the layer will not learn an additive bias. Default: True

Input1: $(*, H_{in1})(*, H_{in1})$ where $H_{in1} = in1_features$ and $*$ means any number of additional dimensions including none. All but the last dimension of the inputs should be the same.

Input2: $(*, H_{in2})(*, H_{in2})$ where $H_{in2} = in2_features$.

Output: $(*, H_{out})(*, H_{out})$ where $H_{out} = out_features$ and all but the last dimension are the same shape as the input.

weight (torch.Tensor) – the learnable weights of the module of shape $(out_features, in1_features, in2_features)$. The values are initialized from $U(-k, k)$, where $k = \frac{1}{\sqrt{in1_features}}$.

bias – the learnable bias of the module of shape $(out_features)$. If bias is True, the values are initialized from $U(-k, k)$, where $k = \frac{1}{\sqrt{in1_features}}$.

Return the extra representation of the module.

Runs the forward pass.

Resets parameters based on their initialization used in `__init__`.